Meilenstein am 29.10.2026: aktueller Prüfstand

Aktualisierung: 28.10.2026 · **Quellcodebasis:** `02eb79f8`

Aktualisierter Prüfstand: Arbeitstag 28.10.2026 · **Ausgangscommit:** `02eb79f8` Der Bericht dokumentiert den technischen Entwicklungsstand. Eine schulische Freigabe wird nicht vorweggenommen.

Bereich	Aktueller Nachweis	Noch offen
Projektstart	Entwicklungsstand seit 22.10.2026, Projektauftrag aktualisiert	Projektziel selbst erläutern
Anforderungen	FR-01-FR-18 erhalten, neue GUI-/Audio-/Logo-Ergänzungen getrennt dokumentiert	Mit Lehrkraft abgleichen
Product Backlog	PB-01-PB-18 erhalten, PB-19-PB-24 ergänzen Zusatzumfang und Abnahme	Priorität und offenen Status verfolgen
Klassenentwurf	Game, Board, Player, Move sowie ConsoleUI, MatchSession, GymMove und App beschrieben	Unterschiede am Code erklären
Menü	Geführter Start, Fortsetzen, Ergebnis und Revanche	Am Schulrechner vorführen
GitHub	Öffentliches Repository mit acht PDF-/PPTX-Dateien, aktueller Zusatzstand in Draft-PR #1	Kein automatischer Merge nach main
Automatische Qualität	Sechs C++-/GUI-Jobs und Dokumentenjob am Ausgangsstand erfolgreich	Zielgeräteprüfung ergänzen
KI-Dokumentation	Historisches Register erhalten, heutige Aufträge und Fehler ergänzt	Kein vollständiger technischer Chat-Export behauptet
Vortrag	PPTX/PDF und Leitfaden auf Konsole plus 2D-GUI aktualisiert	PowerPoint- und Beamerprobe

Heutiger Fortschritt

Die JK-AXIS-Gestaltung und die schülergerechten Kommentarrichtlinien sind dokumentiert. Das Terminalspiel erhielt eine Trainings-Arena und einen durchgängigen Menüablauf. Der Entwicklungsbranch ergänzt eine echte SFML-2D-Anwendung, sicheren Dateiersatz, GUI-unabhängige MatchSession-Tests, Musikschalter, Schullogo und ein Skript für die nacheinander ausgeführte Vorführung.

Wikimedia antwortete beim Logo-Download mit HTTP 429. Das Projekt verwendet einen dokumentierten alternativen Bildhost und enthält inzwischen eine skalierte JPEG-Kopie. Der zuletzt gemeldete F5-Fehler betraf CMake im Suchpfad. Die Build-Aufgaben wurden korrigiert. Der

erfolgreiche lokale Wiederholungsversuch ist noch nicht belegt.

Nachweise

- Aktueller C++-/GUI-Lauf und Dokumentenlauf.
- Gesicherter CI-Snapshot.
- Entwicklungsprotokoll, Architektur und Dokumentenaudit.

Die ältere sieben Seiten lange PDF-Prüfung beschreibt den Stand vom 25.10.2026. Der aktualisierte PDF-Nachtrag stellt den heutigen Stand vor diesen historischen Teil. Die dortige Aussage „GitHub-Veröffentlichung offen“ gilt nur für den damaligen Prüfzeitpunkt.

Vor dem Unterricht

Beide F5-Profile auf dem vorgesehenen Rechner starten. Eine Runde mit Revanche und Speichern/Laden testen. Schullogo und Audio prüfen. Präsentation und Codeerklärung proben. Beobachtungen erst nach dem tatsächlichen Test in die manuellen Pläne eintragen.

Historischer Anhang

Aktualisierung: 28.10.2026 · **Quellcodebasis:** `02eb79f8`

Die folgenden Originalseiten dokumentieren den damaligen Planungs- und Lernstand vom 22.-25.10.2026. Sie bleiben als Entwicklungsgeschichte erhalten. Ihre Aussagen über fehlende GUI, offene GitHub-Veröffentlichung, ältere Menüs, Befehle und frühere Teststände sind keine aktuellen Anweisungen.

Für den Stand vom 28.10.2026 gelten der vorangestellte Nachtrag, die aktualisierte README und die Dateien unter docs/ im Entwicklungsbranch quality/gameplay-persistence-2026-10-28. Die alten 3D-Bilder sind Designstudien. Die tatsächlich ergänzte Oberfläche ist zweidimensional.

Gedruckte Seitenzahlen und Design-IDs im Anhang gehören zur ursprünglichen Fassung. Vorhandene Sprungziele werden beim Zusammenführen soweit vom PDF-Format unterstützt übernommen. Der neue PDF-Seitenbaum bietet zusätzlich einen direkten Einstieg in den Archivteil.

BERND BLINDOW GRUPPE
SCHULE AUSBILDUNG STUDIUM

MEILENSTEIN / 29. OKTOBER 2026

Vier Gewinnt

Gym Edition

Projektprüfung & Product Backlog
Anforderungen · UML · Konsolenmenü · GitHub-Start

C++17 / OBJEKTORIENTIERT / KONSOLE / 2–4 SPIELENDE

Status am 25.10.2026

Belegt ist der Entwicklungsstand, nicht eine abgeschlossene Schulabnahme.

4 Bereiche bereits belegt

Projektdateien, 19-seitige Anforderungen, UML mit vier Typen und ein startfähiges Konsolenmenü.

Product Backlog neu erstellt

18 priorisierte Backlog Items mit Akzeptanzkriterien, Quelle und ehrlicher Unterscheidung zwischen Implementierung und Test.

GitHub-Veröffentlichung noch offen

Ein lokales Git-Repository wird vorbereitet. Im verbundenen GitHub-Konto wurde kein passendes Remote-Repository gefunden.

Agenda & Meilensteinprüfung

Direkte Navigation zu den Nachweisen. Der Status bildet den tatsächlichen Stand vom 25.10. ab.

01	Projektstand & Anforderungen	S. 02 ›
02	Product Backlog · Muss-Kriterien	S. 03 ›
03	Product Backlog · Weitere Punkte	S. 04 ›
04	Architektur & Konsolenmenü	S. 05 ›
05	Qualität & Testnachweise	S. 06 ›
06	GitHub-Übergabe & nächste Schritte	S. 07 ›

ABGLEICH MIT QUELLCODE UND DOKUMENTATION

ARBEITSPAKET	STATUS	NACHWEIS / BEWERTUNG
Projektstart	BELEGT	Dateien und Entwürfe vom 22.10.2026
Anforderungen	BELEGT	Anforderungsanalyse, 19 Seiten, FR-01–18
Product Backlog	NEU / ERSTELLT	PB-01–PB-18 in `docs/PRODUCT_BACKLOG.md`
GitHub-Repository	OFFEN	8 zugängliche Repos geprüft; kein passender Treffer
Klassenentwurf	BELEGT	UML, 6 Seiten; Game, Board, Player, Move
Konsolenmenü	GETESTET	`Game::run()`, Auswahl 0–6; Menü-Smoke-Test

Abgabeinterpretation

Vier Bereiche waren schon anhand von Dateien belegbar. Das Backlog wurde ergänzt. Der letzte offene Pflichtpunkt ist ein nachweisbarer GitHub-Push.

Product Backlog · Kernlogik

PB-01 bis PB-09 | MUST: Pflichten aus der Anforderungsanalyse, priorisiert nach Spielablauf.

ID	PRIO	USER STORY / THEMA	AKZEPTANZKRITERIUM	PRÜFSTAND
01	MUST	Startmenü (FR-01)	Menüpunkte 0–6; Fehler abfangen.	Smoke-Test OK
02	MUST	Spielerzahl (FR-02)	Genau 2/3/4 erlauben.	Code; Test offen
03	MUST	Namen (FR-03)	Nichtleer und eindeutig.	Code; Test offen
04	MUST	Feldmaße (FR-04)	2→7×6, 3→9×8, 4→11×10.	Board-Test OK
05	MUST	Stein setzen (FR-05)	Unterste freie Zelle wählen.	Teiltest OK
06	MUST	Spielerwechsel (FR-06)	Nur nach gültigem Zug.	Code; Test offen
07	MUST	Sieg (FR-07)	Alle vier Achsen prüfen.	Board-Test OK
08	MUST	Remis/Ende (FR-08/13)	Voll ohne Sieg; Ende ausgeben.	Code; Test offen
09	MUST	Fehleingabe (FR-09)	Brett/Zug unverändert.	Teiltest OK

Statusmethodik

„Code“ bedeutet nur im Quelltext nachweisbar. „Board-Test OK“ bezieht sich auf die tatsächlich durchlaufenen `assert`-Prüfungen, nicht auf eine vollständige Lehrkraft-Abnahme.

Product Backlog · Lieferung

PB-10 bis PB-18 | Kernumfang, Dokumentationspflicht und optionale Gym-Funktionen.

ID	PRIO	USER STORY / THEMA	AKZEPTANZKRITERIUM	PRÜFSTAND
10	MUST	Speichern (FR-10)	Formatversion, Personen, Feld und Zuglage.	Code; Test offen
11	MUST	Laden (FR-11/14)	Defekte Datei verändert aktive Partie nicht.	Code; Test offen
12	MUST	Highscore (FR-12)	Sieg +1, dauerhaft, sortiert.	Code; Test offen
13	MUST	GitHub (FR-15)	Remote + sichtbare Commits.	Noch offen
14	MUST	KI-Prompts (FR-16)	Prompt-Einsatz prüfbar dokumentieren.	Vorlage erstellt
15	MUST	Vorführung (FR-17/18)	Handout, Demo und Code erklären.	In Vorbereitung
16	SHOULD	ASCII/Hilfe (EX-01/05)	Symbole klar; Hilfefunktion ohne Zug.	Code; Sichttest offen
17	COULD	Vorschau/Undo/S (EX-02–04)	Keine unzulässige Zustandsänderung.	Teilweise geprüft
18	COULD	Info/Animation (EX-06/07)	Ohne Animation spielbar.	Code; Sichttest offen

Definition of Done

Akzeptanzkriterium erfüllt + tatsächlicher Test protokolliert + Dokumentation aktualisiert + nachvollziehbarer Commit. Nur programmierter Quellcode reicht nicht für die Freigabe.

Klassenentwurf & Prototyp

Ist-Modell aus `include/*.h` und `src/*.cpp`; keine echte grafische GUI.

Game

Steuerung, Menü, Spielablauf, Spielerwechsel, Dateien & Highscores

Board	Spielfeld und Gewinnlogik
Player	Spielername und Symbol
Move	Zug-Historie für Undo

PROGRAMMSTART

main.cpp → Game game; → game.run(); → Game::play()

TATSÄCHLICH IMPLEMENTIERTES KONSOLENMENÜ

VIER GEWINNT | GYM EDITION | C++17

1 Neues Spiel
2 Spieleranzahl festlegen
3 Namen eingeben
4 Spiel speichern
5 Spiel laden
6 Highscore
0 Beenden

Abgrenzung: Mockup ≠ Implementierung

Die blaue 3D-Spieloberfläche ist ein visueller Entwurf. Belegbar lauffähig ist das textbasierte C++17-Programm mit ASCII-Hanteln.

Build- & Testnachweise

Erneute praktische Prüfung der vorhandenen Dateien am 25.10.2026.

ID	TEST	METHODE	ERGEBNIS
Q-01	C++17-Projekt	g++ 14.2.0; `-Wall -Wextra -pedantic`	Bestanden, 0 Warnungen
Q-02	Board-Testbau	`board_tests.cpp` + `Board.cpp`	Bestanden, 0 Warnungen
Q-03	Automatische Tests	Feldgrößen, Grenzen, 4 Achsen, Undo	Assertions bestanden
Q-04	Menü-Smoke-Test	Eingaben `6` / `8` / `0`	Bestanden

NOCH AUSSTEHENDE ABNAHMETESTS

- Komplette Partie: Spielerwechsel, volle Spalte, Sieger-/Remiserkennung.
- Persistenz: Speichern, Neustart, Laden, beschädigte Datei und Zustandsschutz.
- Dauerhafter Highscore: +1 pro Sieg, korrekte Sortierung, erneuter Programmstart.
- Praxisprüfung unter Windows / VS Code sowie GitHub-Remote-Nachweis.

Transparente Beweislage

Die bereits vorhandene Testplandatei enthielt leere Ist-/OK-Felder. Diese werden nicht nachträglich als bestanden ausgegeben. Der konkrete Build-Nachweis liegt in `docs/QUALITY_AND_TESTS.md`.

GitHub & Abschluss am 29.10.

Das Projektpaket enthält bereits ein lokales Git-Repository mit einem Start-Commit.

WAS IST IETZT NOCH ZU TUN?

01 / Neues leeres privates Repository

Auf github.com/new im Konto Julik527 `vier-gewinnt-gym-edition` erstellen. Kein automatisches README erzeugen.

02 / Lokalen Projektordner öffnen

ZIP entpacken und das Projektverzeichnis mit `.git`, `src` und `include` in VS Code öffnen.

03 / Main-Branch hochladen

Remote eintragen und pushen. GitHub-Login gegebenenfalls im Browser bestätigen.

04 / Nachweise kontrollieren

GitHub-Seite öffnen; README, Backlog, UML, C++-Quelldateien und Commit-Hash überprüfen.

TERMINAL (IM ENTPACKTEN PROJEKTSTAMM)

```
git status
git remote add origin https://github.com/Julik527/vier-gewinnt-gym-edition.git
git branch -M main
git push -u origin main
```

Ehrlicher Freigabestatus

Das lokale Git-Repository ersetzt keinen GitHub-Upload. Erst wenn die GitHub-URL erreichbar ist und der Commit sichtbar wird, gilt die Repository-Anforderung als erfüllt.